

SR10 Rapport TD Sécurité

Dans le cadre de notre projet SR10, nous devons répondre à plusieurs exigences en matière de sécurité. Il nous était notamment demandé de corriger au moins trois vulnérabilités présentes sur notre site.

Nous avons ainsi choisi de traiter les vulnérabilités suivantes :

- **Stockage des mots de passe en clair dans la base de données** : nous avons mis en place un hachage sécurisé des mots de passe.
- **Injection SQL** : nous avons sécurisé nos requêtes en utilisant des requêtes préparées.
- **Attaques par force brute** : un mécanisme de limitation du nombre de tentatives de connexion a été intégré.
- **Contrôle de session et gestion des accès** : nous avons renforcé les vérifications d'authentification et mis en place des restrictions d'accès en fonction des rôles des utilisateurs.

I. Hash du mot de passe

La vulnérabilité la plus préoccupante, en termes d'importance et surtout en matière de protection et de confidentialité des données, concerne notamment les mots de passe.

Lorsqu'un nouvel utilisateur était créé, son mot de passe était enregistré en clair dans la base de données. Cela représentait une faille de sécurité majeure : si la base de données fuyait, n'importe qui pourrait accéder aux identifiants et mots de passe des utilisateurs. De plus, cela ne garantissait aucune confidentialité des données. L'entreprise n'a pas à connaître les mots de passe de ses utilisateurs.

La solution mise en place a donc été de hacher les mots de passe. Désormais, lorsqu'un nouvel utilisateur crée son compte, ce n'est plus son mot de passe en clair qui est enregistré dans la base de données, mais un hash de celui-ci, généré à l'aide de la méthode **bcrypt.hash()**.

Lors de la connexion, l'utilisateur saisit son mot de passe, qui est immédiatement haché, puis comparé (**bcrypt.compare()**) au hash stocké dans la base. Si les deux correspondent, cela signifie que le mot de passe est correct.

L'intérêt du hachage réside dans le fait qu'il est facile de calculer le hash d'un mot de passe, mais quasiment impossible de retrouver le mot de passe d'origine à partir du hash. Ainsi, même en cas de fuite de données, les mots de passe des utilisateurs restent protégés.

Le principal risque du hachage est la collision : cela signifie que deux mots de passe différents produisent le même hash. Une telle situation pourrait être extrêmement grave, c'est pourquoi il est essentiel de choisir une fonction de hachage robuste, comme bcrypt, qui est spécifiquement conçue pour minimiser ce type de vulnérabilité.

33	UpdatedTest	UpdatedPrenom	create@test.fr	password	candidat	NULL	alive
34	Petit	Alice	alice@gmail.com	AliceP	candidat	attente	alive
80	Boto	Lewis	lewis@ex.com	RAI	candidat	NULL	alive
81	Vives	Ninon	n@gmail.com	nvives	candidat	attente	alive
82	Vives	Martin	m@gmail.com	mvives	candidat	NULL	alive

Capture d'écran de la BDD avec les mots de passe en clair

6	Botokey	Lewis	lewis@gmail.com	\$2b\$10\$f0iK7LADBXEXXZ4yGMDfMuujD9rCeEUmMRL0Zh.NRRX...	admin	validé
8	Valet	Martin	martinv@gmail.com	\$2b\$10\$xU0boxXc0uUtUO1BnTmDu.fSXUQzLrva/9fxHHk1zup...	recruteur	validé
33	UpdatedTest	UpdatedPrenom	create@test.fr	\$2b\$10\$pbMqNba4qLC1.FUAqA71TOJXzIHKrlqkYdXnxRsNQpG...	candidat	NULL
34	Petit	Alice	alice@gmail.com	\$2b\$10\$kYn4fYAv4TBRNH.53sS/M.q7XSCswkYjeskZChtQOpq...	candidat	attente
80	Boto	Lewis	lewis@ex.com	\$2b\$10\$oIRlJly4aHuXC8yUar6suLKDKhvcBBBQloebckI/5z...	candidat	NULL
81	Vives	Ninon	n@gmail.com	\$2b\$10\$30ddgdDSSjl0tv9zHqijh.1ZI.snBQHvQOHCKl6Jzua...	candidat	attente
82	Vives	Martin	m@gmail.com	\$2b\$10\$afvcwAcJCmEpEUN4g4Dnpu9Tob38mqJ2t4xKBdOXz/v...	candidat	NULL
83	Botokey	Lewis	lewis.botokey@etu.utc.fr	\$2b\$10\$sSj2Am4XGbT9t1Tb2oROMe8TSOmCIT0d7DWGBYlu.NA...	admin	NULL
89	MDP	Test	lol@lol.com	\$2b\$10\$vjztJ3T6gyPF5oS6i/Ml/.8nF91faDY0948gYvLnIR3...	candidat	NULL

Capture d'écran de la BDD avec les mots de passe haché

II. Injection SQL

L'**injection SQL** est encore aujourd'hui l'une des attaques les plus courantes sur le Web. Elle s'effectue via un formulaire — par exemple, une barre de recherche — dans lequel une ligne de texte malveillante peut être saisie afin d'afficher des données non autorisées, voire de supprimer complètement une table.

La vulnérabilité survient lors de l'écriture des requêtes SQL.

```

if (metier) {
  sql += ` AND Fiche_de_Poste.type_metier LIKE '${metier}'`;
}

```

Requête SQL dangereuse

Voici un extrait de notre code. Dans cette requête, on constate que les variables sont directement intégrées à la chaîne SQL, sans séparation. Par conséquent, si l'on tape dans la barre de recherche : **a' OR '1' = '1**, cela affichera toutes les offres, y compris celles qui sont non publiées ou expirées.

Liste des Offres d'Emploi

Mots-clés (intitulé, lieu, description...) **a' OR '1' = '1** Salaire min Date Statut du poste

Salaire max

Rechercher

Manager d une team de développeur 2025-06-12	publiée	CV + lettre de motivation	Visualiser l'Offre	Postuler
Test injection SQL 2025-06-10	expirée	CV + Permis de conduire	Visualiser l'Offre	Postuler
Test injection SQL 2025-06-10	publiée	Attention à l'attaque	Visualiser l'Offre	Postuler
test xss 2 2025-06-10	publiée	dsf	Visualiser l'Offre	Postuler

Injection SQL réussie, une offre expirée est affiché

Les conséquences peuvent être bien plus graves. Avec ce type d'attaque, un utilisateur malveillant peut également supprimer des données ou en modifier le contenu.

Pour se protéger contre ce genre d'attaque, il est essentiel de **préparer les requêtes**. Cela consiste à séparer la structure de la requête des variables. Les variables sont représentées par des points d'interrogation dans la requête, et on exécute celle-ci avec la méthode **db.query([requête], [variables])**

```

if (metier) {
  sql += ` AND Fiche_de_Poste.type_metier LIKE ?`;
  params.push(`%${metier}%`);
}

```

```
db.query(`SELECT * FROM Utilisateur WHERE email = ?  
| AND motDePasse = ?`,[email, password],
```

Requête SQL bien préparé, peut être utilisé

III. Attaque par Brute force

Dans notre application, nous avons anticipé les attaques par **brute force** lors de la connexion utilisateur. Le code du contrôleur **usersController.js** implémente un mécanisme de limitation des tentatives de connexion.

En effet, avant cela on pouvait effectuer autant de tentatives de connexion qu'on pouvait donc très vulnérable à une attaque par Brute Force, qui consiste à tester toutes les combinaisons de mot de passe possibles.

Fonctionnement de la protection

- À chaque tentative de connexion échouée, un **compteur** est incrémenté pour l'adresse mail concernée.
- Après **5 tentatives** infructueuses, le compte est verrouillé pendant 5 minutes (lockUntil).
- Un message d'erreur spécifique est retourné à l'utilisateur en cas de verrouillage.

Extrait du code concerné :

```
if (!loginAttempts[email]) {
  loginAttempts[email] = { attempts: 0, lockUntil: null };
}
const userAttempts = loginAttempts[email];
// Vérifier si Le compte est verrouillé
if (userAttempts.lockUntil && userAttempts.lockUntil > Date.now()) {
  throw new Error(`Compte verrouillé jusqu'à ${new Date(
    userAttempts.lockUntil).toLocaleString()}. Veuillez réessayer plus tard.`);
}
...
if (!isMatch) {
  userAttempts.attempts +=1;
  if (userAttempts.attempts >= 5) {
    userAttempts.lockUntil = Date.now() + 5 * 60 * 1000; // verrouillage pour 5
    minutes
    userAttempts.attempts = 0;
    return res.status(403).json({ message: "Compte verrouillé après 5 tentatives
    échouées." });
  }
  throw new Error("Mot de passe incorrect. Tentatives restantes : " + (5 -
    userAttempts.attempts));
}
```

Message affiché après une mauvaise tentative :

```
{"message": "Erreur serveur.", "error": "Mot de passe incorrect. Tentatives restantes : 4"}
```

Après la 5ème tentative :

```
{"message": "Compte verrouillé après 5 tentatives échouées."}
```

Si on tente de se connecter une 6ème fois :

```
{"message": "Erreur serveur.", "error": "Compte verrouillé jusqu'à 14/06/2025 10:22:41. Veuillez réessayer plus tard."}
```

IV. Contrôle de session et gestion d'accès

Pour garantir la sécurité et la confidentialité des données, notre application met en place un **contrôle strict des sessions et des droits d'accès** selon le **rôle** de l'utilisateur (candidat, recruteur, administrateur).

Fonctionnement

- Vérification de session : À chaque requête, le serveur vérifie si l'utilisateur est authentifié grâce à la session.
- Rôles et restrictions :
 - Certaines pages sont accessibles uniquement aux administrateurs (ex : gestion des organisations, validation des recruteurs, suppression d'utilisateurs...).
 - D'autres pages sont réservées aux recruteurs (ex : création d'offres, gestion des fiches de poste...).
 - D'autres pages sont réservées aux candidat (ex : postuler, candidat/candidature, ...)
 - Les pages publiques (connexion, création de compte) restent accessibles à tous.
- Redirections et messages d'erreur :
 - Si un utilisateur tente d'accéder à une page sans avoir le bon rôle, il est redirigé ou voit un message d'erreur (« Accès réservé à l'administrateur », « Accès réservé au recruteur », etc.).
 - Un utilisateur non connecté est systématiquement redirigé vers la page de connexion.

Exemple de logique (extrait du code) :

```
app.all("*", function (req, res, next) {  
  
  const nonSecurePaths =  
    ["/", "/utilisateurs/new_user", "/authentification", "/utilisateurs/creation_compte"];  
  //List des urls non sécurisées  
  // Routes réservées aux admins  
  const adminPaths = [  

```

```

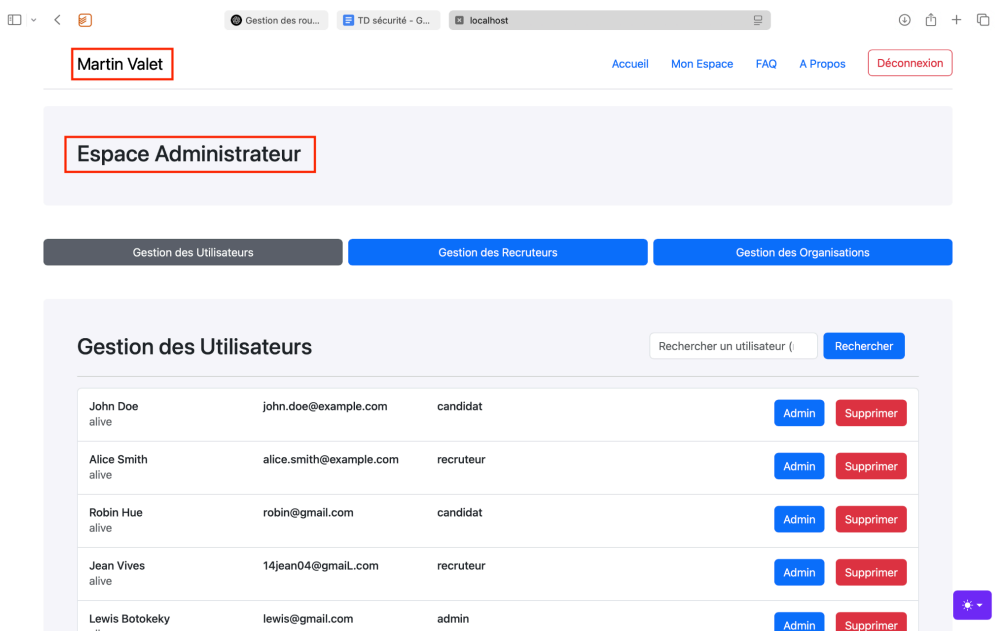
    "/utilisateurs/recruteur",
    "/utilisateurs/valider_recruteur",
    "/utilisateurs/delete",
    "/utilisateurs/set_admin",
    "/organisations",
    "/organisations/accepter",
    "/organisations/supprimer"
  ];
  // Routes réservées aux recruteurs
  const recruteurPaths = [
    "/offre",
    "/offre/new_offre",
    "/offre/creation_offre",
    "/utilisateurs/espace_recruteur",
    "/fiche_de_poste",
    "/fiche_de_poste/new_fiche_de_poste"
  ];

  if (nonSecurePaths.includes(req.path)) return next();

  // Accès admin uniquement
  if (adminPaths.some(adminPath => req.path === adminPath)) {
    if (session.isConnected(req.session, "admin")) return next();
    return res.status(403).render("error", { message: "Accès réservé à l'administrateur.", error: {} });
  }
  // Accès recruteur uniquement
  if (recruteurPaths.some(recruteurPath => req.path === recruteurPath)) {
    if (session.isConnected(req.session, "recruteur")) return next();
    return res.status(403).render("error", { message: "Accès réservé au recruteur.", error: {} });
  }
  // Accès général (utilisateur connecté)
  if (session.isConnected(req.session)) return next();
  // Sinon, non authentifié
  return res.redirect("/");
});

```

Par exemple, un recruteur (ici Martin Valet) pouvait accéder à la page réservée à l'administrateur simplement en tapant le bon URL (<http://localhost:3000/utilisateurs/>).



id_user	nom	prenom	email	motDePasse	role	role_recruteur	etat_compte	siren
1	Doe	John	john.doe@example.com	\$2b\$10\$DViMYYRNFOp3jRf89dwpSuOPmK0X99tOPJR0fsfZeS...	candidat	NULL	alive	NULL
2	Smith	Alice	alice.smith@example.com	\$2b\$10\$DViMYYRNFOp3jRf89dwpSuOPmK0X99tOPJR0fsfZeS...	recruteur	validé	alive	123456789
3	Hue	Robin	robin@gmail.com	\$2b\$10\$mwH3B9jWnQgN4LaBFxfFOQn42MX1nN2Ktug1Cl6xN...	candidat	attente	alive	123456789
4	Vives	Jean	14jean04@gmail.com	\$2b\$10\$w29EW9dcccC4QlqhsWD42eX.J1teQF9xkMFSI.5gZlu...	recruteur	validé	alive	25459458
6	Botokey	Lewis	lewis@gmail.com	\$2b\$10\$IOIK7LADBXEXX4yGMDfMuuJd9rCeUmMRL0Zh.NRRX...	admin	validé	alive	25459687
8	Valet	Martin	martinv@gmail.com	\$2b\$10\$xU0boxXc0uUUO1BnTmDu.fSXUQzLrva/9fxHHk1zup...	recruteur	validé	alive	25459458
33	UpdatedTest	UpdatedPrenom	create@test.fr	\$2b\$10\$pbMqNBa4qLC1.FUAqA71TOJXzIHkRqkYdXnxRsNQpG...	candidat	NULL	alive	NULL
34	Petit	Alice	alice@gmail.com	\$2b\$10\$Yn4fYAv4TBRNH.53sS/M.q7XSCswkYjeskZChtQOpq...	candidat	attente	alive	NULL
80	Boto	Lewis	lewis@ex.com	\$2b\$10\$/oIRIJy4aHuXC8yUar6suLKDkhvBBBBQloebcKI/5z...	candidat	NULL	alive	NULL
81	Vives	Ninon	n@gmail.com	\$2b\$10\$30ddgdDSSjI0tv9zHqjih.1Zl.snBQHVQOHckl6Jzua...	candidat	attente	alive	NULL
82	Vives	Martin	m@gmail.com	\$2b\$10\$afvcwAcJCmEpEUN4g4Dnpu9Tob38mqJ2t4xKBdOXz/v...	candidat	NULL	alive	NULL
83	Botokey	Lewis	lewis.botokey@etu.utc.fr	\$2b\$10\$SsSj2Am4XGb79t1Tb2oROMe8TSOmCIT0d7DWGBYlu.NA...	admin	NULL	alive	123456789
89	MDP	Test	lol@lol.com	\$2b\$10\$vjztJ3T6gyPF5oS6i/M/.8nF91faDY0948gYvLnIR3...	candidat	NULL	alive	NULL
90	t	h	test@hash.com	\$2b\$10\$mVaDqj8Wvb7GgEck0.2EYuAjra9UwKRPxK2Hp6flfxr...	admin	NULL	alive	NULL
91	test	candidat	testcandidat@mail.com	\$2b\$10\$AZhoBIL/B9m9sl6UctXAf.VcDZRCdnMTLAnyLV4zib...	candidat	NULL	alive	NULL

Après la mise en place : Tentative d'accès à une page réservée à l'administrateur

